

EXAMEN PARCIAL

APELLIDOS Y NOMBRES			
CURSO	Programación para Dispositivos Móviles	SEMESTRE	Sexto
DOCENTE	Josue Miguel Flores Parra	FECHA: 19-05-2026	
DURACIÓN	3 horas.		

Examen Parcial: Juego de Colores

Desarrolla una aplicación utilizando **Kotlin, Jetpack Compose y Android Studio**. El juego pondrá a prueba tu capacidad para desarrollar interfaces modernas, manejar estados, navegación, arquitectura MVVM y lógica interactiva.

 Objetivo:

Crear un juego donde el usuario debe presionar el botón que coincida con el color mostrado en pantalla. El objetivo es obtener la mayor cantidad de aciertos posibles en un tiempo límite.

La aplicación debe utilizar:

- Jetpack Compose
- Navigation Compose
- Arquitectura MVVM
- Manejo de estados
- Componentes visuales modernos

 ¿Qué debe tener la aplicación?**1. Fragmento de bienvenida (WelcomeScreen)**

- Título del juego.
- Mensaje de bienvenida y reglas del juego.
- Botón **“Iniciar Juego”**.
- Diseño libre utilizando componentes Compose.
- Puede incluir imágenes, iconos o animaciones simples.

2. Fragmento del juego (GameScreen)

- Un área visual que muestre un color aleatorio (rojo, verde, azul, amarillo, etc).
- Botones de colores para responder.
- Temporizador de 30 segundos. Investigar: **LaunchedEffect, corrutinas (delay)**.
- Puntaje actual. Cada acierto suma un punto.
- Cambio automático del color después de cada respuesta correcta.
- Feedback visual o textual cuando el usuario:

- acierta
- se equivoca
- Muestre el puntaje actual y el tiempo restante.
- El juego finaliza al agotar el tiempo.

3. Fragmento de resultados (ResultScreen)

- Puntaje final de la partida.
- Puntaje más alto histórico (**Usar SharedPreferences**).
- Historial de partidas utilizando **LazyColumn**. El historial de puntajes sin persistencia. Puede solo visualizar el historial de las partidas de la sesión.
- Mostrar al menos:
 - puntaje
 - fecha o número de partida
- Botón para volver a jugar.

Arquitectura requerida

La aplicación debe implementar:

- ViewModel
- MutableStateFlow, MutableState y/o MutableListState
- collectAsState
- Navegación utilizando NavHost
- Separación entre UI y lógica de negocio (MVVM)

Funcionalidades Adicionales (elige al menos una, es **obligatoria**):

Seleccionar e implementar únicamente UNA de las siguientes opciones:

-  Agrega sonidos al juego (por ejemplo, cuando acierta, se equivoca o música de fondo).
-  Guarda el historial de puntajes en una base de datos usando **Room**, recupera el puntaje más alto con una consulta a la base de datos usando **Room**.
-  Aplica animaciones utilizando compose a los botones o colores que aparecen.
-  Implementa **niveles de dificultad**. Ejemplos:
 - aumentar velocidad
 - aumentar cantidad de colores
 - Efecto Stroop: Muestra un texto que dice “Rojo” en color azul, El jugador debe presionar el color **del texto**, no el que dice la palabra, aumentar el número de colores conforme se avanza,
 - penalizaciones

¿Qué se debe practicar?

- Jetpack Compose
- Navigation Compose
- Arquitectura MVVM
- Manejo de estados
- MutableStateFlow, MutableState y/o MutableListState
- collectAsState
- Navegación entre pantallas
- Temporizadores (**LaunchedEffect**, **corrutinas (delay)**)
- Lógica condicional
- Diseño de interfaces modernas
- Uso de LazyColumn
- Organización del código
- Persistencia básica de datos
- Uso correcto de componentes Compose

Importante

- No utilizar XML.
- Toda la interfaz debe desarrollarse con Jetpack Compose.
- Implementar navegación con Compose.
- La lógica principal no debe colocarse directamente en las pantallas.
- El proyecto debe compilar correctamente.
- El código debe estar organizado y comentado.

Tips útiles:

- Usa recursos de cadenas en strings.xml.
- Usa recursos de colores en colors.xml.
- Puedes usar una función para generar el color aleatorio.
- Para el temporizador, revisa la clase CountdownTimer.
- Para el puntaje más alto histórico, revisa SharedPreferences
- Puedes usar Scaffold para organizar pantallas.
- Usa remember, mutableStateOf o StateFlow.
- Puedes utilizar LaunchedEffect para temporizadores.
- Puedes utilizar Random.nextInt() para generar colores aleatorios.
- Utilizar strings.xml para textos visibles al usuario.
- Utilizar ui/theme/Color.kt o archivos del tema para manejo de colores.
- Usa LazyColumn para mostrar historial.
- Puedes usar Material 3 para mejorar la interfaz..